A private LoRa sync word on the raw channel, without breaking LoRaWAN

Tested on a Dragino DLOS8N (SX1302, OpenWrt 18.06, `mips_24kc`), Semtech `sx1302_hal` V2.1.0. Should apply to any SX1302 gateway — LPS8v2, RAK7268 and relatives — since everything here is chip-level.

The short version: **the SX1302 has four RX sync word register pairs, not one.** Three serve the shared multi-SF block, and one belongs to `chan_Lora_std` alone. So the raw channel can carry an arbitrary sync word *while* the eight LoRaWAN channels keep 0x34 — simultaneously, no switching involved.

The common belief that the sync word is chip-wide is only three-quarters true, and the remaining quarter is the useful one.

What the silicon actually offers

Register pair	Address	Applies to
`SF5_PEAK1/2`	`0x588A/0x588B`	all 8 multi-SF channels, SF5 only
`SF6_PEAK1/2`	`0x588C/0x588D`	all 8 multi-SF channels, SF6 only
`SF7TO12_PEAK1/2`	`0x588E/0x588F`	all 8 multi-SF channels, SF7–SF12
`LORA_SERVICE_PEAK1/2`	`0x5B2E/0x5B2F`	**only `chan_Lora_std`**

What is not possible: giving *one of the eight* multi-SF channels its own sync word. Those eight share a demodulator block whose sync word is split by spreading factor only — there is no per-IF-channel register. No amount of software changes that.

Time-multiplexing does not rescue it either. I measured the switch cost: 295 µs to retune the multi-SF pair, 590 µs there and back — comfortably inside the 2050 µs sync window at SF7. It still fails, for a structural reason: *there is no trigger.* Packet detection **is** the sync word comparison. By the time anything is observable, the sync symbols have been evaluated and the packet discarded. Blind duty-cycling would be a zero-sum split degrading both sides.

The value range is the full byte

A sync word `0xHL` rides in the two sync symbols of the preamble. The SX1302 stores each symbol as `symbol_value / 4`, i.e.

```
peak_pos = nibble * 2        0x34 -> peak1 = 6,  peak2 = 8
                             0x12 -> peak1 = 2,  peak2 = 4
                             0x55 -> peak1 = 10, peak2 = 10
```

The field is 5 bits. The register table declares it signed, but that only affects reads — `lgw_com_rmw()` masks unsigned on write, so values up to 30 go through and **all 256 sync words 0x00–0xFF are reachable**. The 0x12/0x34 restriction is purely the `lorawan_public` boolean in the HAL.

Why interpose rather than replace the HAL

Dragino moved the SX1302 chip reset *into* `libsx1302hal.so` — it links against `libgpio.so`, and `/etc/init.d/lora_gw` calls no reset script for sx1302. Dropping in an upstream-built HAL loses that reset.

So instead of replacing the library, replace exactly one function:

```
int sx1302_lora_syncword(bool public, uint8_t lora_service_sf);
```

No structs cross that boundary, so the ABI is untouched and the stock packet forwarder keeps running unmodified. Registers are addressed **by absolute address** through `lgw_com_rmw()`, not by register-table ID, so it does not matter whether the vendor's register enumeration matches upstream's.

One MIPS detail that costs an hour if you hit it: MIPS resolves GOT entries eagerly at load time, so an `LD_PRELOAD` library with an undefined symbol dies with `symbol not found` before the HAL is loaded. The shim therefore carries a `DT_NEEDED` on `libsx1302hal.so`, produced by linking against a throwaway stub.

Three traps worth knowing about

1. The file you edit is not the file that is read

Editing `/etc/lora/global_conf.json` is pointless. `init_board()` in `/etc/init.d/lora_gw` runs `/usr/bin/generate-config.sh` on **every** start, and that copies `/etc/lora/cfg-302/EU-global_conf.json` over it. Edit the **template**.

2. procd overwrites your `LD_PRELOAD`

`procd_set_param env LD_PRELOAD=…` silently loses: procd sets `LD_PRELOAD=/lib/libsetlbf.so` itself for line buffering. Use a wrapper that appends *after* procd:

```
#!/bin/sh
SHIM=/usr/lib/libsx1302syncword.so
[ -n "$LD_PRELOAD" ] && LD_PRELOAD="$LD_PRELOAD:$SHIM" || LD_PRELOAD="$SHIM"
export LD_PRELOAD
exec /usr/bin/fwd "$@"
```

3. LDRO is never set at BW500

The HAL derives it from

```
#define SET_PPM_ON(bw,dr) (((bw == BW_125KHZ) && ((dr == DR_LORA_SF11) || (dr ==
DR_LORA_SF12))) \
                          || ((bw == BW_250KHZ) && (dr == DR_LORA_SF12)))
```

— true only for BW125 with SF11/SF12 and BW250 with SF12, so **never for BW500**. Ebyte modules transmit BW500 with LDRO = 1. With the wrong LDRO the header locks and `HeaderValid` fires, but *every* payload arrives with a CRC error. It looks like "almost working", which makes it the most expensive trap of the lot. Override is register `0x5B22`, bits 4–5.

Reading and writing registers while the forwarder runs

Sweeping a value normally means restarting the forwarder for every candidate. It doesn't have to: the SX1302 has **no page register** (unlike the SX1301), so an access is fully described by its address, and each access is a single `ioctl(SPI_IOC_MESSAGE)` that the kernel serialises against the forwarder's own transfers. A small helper on `/dev/spidev1.0` is therefore safe alongside a running `fwd`.

Frame format, taken from `libloragw/src/loragw_spi.c`:

```
write   [mux, 0x80 | (addr>>8)&0x7F, addr&0xFF, data]           4 bytes
read    [mux,        (addr>>8)&0x7F, addr&0xFF, 0x00, 0x00]     5 bytes
```

That turned a sixteen-restart hunt into one short measurement window.

Worked example: an Ebyte E22 / E90-DTU

Two things about Ebyte modules that cost me time, both measured rather than read:

The air rate labels are nominal, and the ladder is BW500 throughout. The table circulating online maps them to BW125. That is wrong. Index 2 ("2.4k") is **SF11/BW500**; index 5 ("19.2k") is SF7/BW500. The manual confirms it in passing with *"air data rate 2.4kbps@SF11"* — 2.4 kbps at SF11 only works out at BW500; at BW125 it would be 537 bps.

The sync word is 0x55. Ebyte does not expose it and no manual lists it. It cannot be fully determined with an SX126x receiver, because that only evaluates the *first* of the two sync word register bytes — a sweep there hits on all of 0x58–0x5F. The SX1302 checks both peak positions strictly and resolves it: a live sweep of `peak2` gave packets only at `peak2 = 10`, i.e. **0x55**.

Frequency for the 900 MHz series is `850.125 MHz + channel × 1 MHz`; the factory default channel 18 lands exactly on 868.125 MHz.

The module wraps the payload even in transparent mode. Note that bytes 2–3 are a **checksum over the payload, not a counter** — the same payload produces a byte-identical frame — and that the whitening key is a constant 0x12, not the channel number (the 868 device's channel just happens to be 18 = 0x12; the 433 device uses channel 23 and still whitens with 0x12). Bytes 5–6 carry the sending module's **own** address, which is what makes self-filtering possible:

```
2C 12 87 26 00 FF FF 07 | 42 40 5D 56 3F 22 21
                                  XOR 0x12         -> "PROD-03"


byte 0    0x2C   magic
byte 1    channel number
byte 2-3  xx, xx ^ 0xA1  where xx = (XOR over all payload bytes) ^ 0xA0
byte 4    NETID
byte 5-6  source address of the sending module
byte 7    payload length
```

Result

Gateway configured with `chan_Lora_std` at 868.125 MHz, SF11, BW500, sync word 0x55, LDRO forced to 1:

```
[syncword] LoRa Service LDRO forced to 1 (HAL rule overridden)
[syncword] multi-SF ch0-7: SF5=0x12 SF6=0x12 SF7-12=0x34 | Lora_std (SF11): 0x55
```

Ebyte packet received on **chan 8**:

```
{"chan":8,"freq":868.125000,"datr":"SF11BW500","rssi":-63,"stat":1,
 "size":15,"data":"LBKHJgD//wdCQF1WPyIh"}
```

LoRaWAN is provably unaffected. Regression test with an LA66 USB node (EU868 v1.3, already joined), uplink on 868.500 MHz at DR0:

```
{"chan":2,"freq":868.500000,"datr":"SF12BW125","rssi":-85,"stat":1,
 "data":"QOBiiQEAAAACTE9SQVdOCtSQ9g=="}
```

Decoded: MHDR `0x40` unconfirmed data up, DevAddr `0x018962E0`, FCnt 0, FPort 2, payload `LORAWN`. Only the service modem's register is touched, so the LoRaWAN chain is untouched by construction — and this confirms it in practice.

The transmit direction

Everything above concerns **receive**. If the gateway also has to *reach* the node, a different place in the HAL applies: `sx1302_send()` rewrites the transmit sync word **for every packet**, right before keying the PA, again derived from `lorawan_public`. A downlink therefore always goes out as 0x34.

Interposing `sx1302_send()` is unattractive — its signature carries struct pointers, which would tie you to the vendor's struct layout. Interpose `lgw_com_rmw()` instead and substitute only the four TX registers `0x526D/0x526E` (TX_TOP_A) and `0x546D/0x546E` (TX_TOP_B).

Make it conditional on the transmit frequency, or you will break LoRaWAN downlinks: join accepts and ADR commands would go out on a sync word no end device accepts. `sx1302_send()` writes the frequency (`loragw_sx1302.c:2604-2608`) *before* the sync word (`:2710`), so it is already known when the decision is needed. Those three bytes are 8-bit direct writes, so watch `lgw_com_w()` rather than `lgw_com_rmw()`:

```
freq_reg = freq_hz * 2^18 / 32e6      ->  122 Hz per LSB
```

That resolution separates a raw channel on 868.125 MHz from `chan_multiSF_0` on 868.100 MHz, 25 kHz away. Verified with a real downlink on 868.125:

frequency filter	peak1 / peak2	sync word
`868125000`	10 / 10	**0x55**, substituted
`869525000` (RX2)	6 / 8	0x34, HAL value untouched

When reading those registers back, note the peak field is the **low five bits**: `0x526D = 0xAA` means `0xAA & 0x1F = 10`; the upper bits carry AUTO_SCALE, GAIN and DROP_ON_SYNCH.

Regulatory footnote

A raw channel with BW500 centred on 868.125 MHz occupies 867.875–868.375 MHz and thus overlaps the LoRaWAN channels on 868.1 and 868.3. Permitted inside the same sub-band, but it counts against the same duty cycle budget — and at SF11/BW500 a 16-byte frame is 144 ms of airtime, which under 1 % locks the transmitter for roughly 14 seconds.

Code

Shim, register poker and a one-shot setup script (with `--status` and `--revert`, backing up every file it touches) are here, together with the same change as a patch against `Lora-net/sx1302_hal` V2.1.0 for anyone who prefers to build the HAL themselves:

https://github.com/gerontec/TTN/tree/main/gateway/sx1302_syncword

Cross-built with the OpenWrt 18.06 SDK for `ar71xx/generic`, `mips_24kc`, musl.